Appendix C

Miscellaneous API Updates

This appendix summarizes miscellaneous updates and changes to the APIs since Java 17 that are relevant for the *Java SE 21 Developer Professional Exam (1Z0-830)*. The exam objectives affected are listed below. For more information on the APIs mentioned here, we recommend consulting the *Java SE 21 Platform API Documentation.*

Java SE 21 Developer Professional Exam Objectives	
[1.2] Manipulate text, including text blocks, using String and StringBuilder classes.	*New methods for searching in a string and repeatedly extending a string builder*
[10.1] Implement localization using locales and resource bundles. Parse and format messages, dates, times, and numbers, including currency and percentage values.	*New overloaded method* `of()` *for constructing locales*

C.1 The `String` and `StringBuilder` Classes

Searching in a String

Two new methods have been added to the `String` class since Java 17 that allow searching for a character or a substring in an *open range* `[a, b)`, where the index `a` is *included* but the index `b` is *excluded* in the search.

The code below illustrates finding a character (`'c'`) in an open range in a string (`"abcd"`). At (1), the character `'c'` is found at index `2` in the open range `[2, 3)`. At (2), the character `'c'` is not found in the open range `[3, 3)`, the index returned is `-1`. At (3) and (4), an `StringIndexOutOfBoundsException` is thrown at runtime, as the ranges `[1, 10)` and `[10, 3)` are invalid for a string of length 4, respectively.

Click here to view code image

```
int index;
index = "abcd".indexOf('c', 2, 3);  // (1)  2: substring in the range [2, 3).
index = "abcd".indexOf('c', 3, 3);  // (2) -1: substring not in the range [3, 3).
index = "abcd".indexOf('c', 1, 10); // (3) StringIndexOutOfBoundsException
index = "abcd".indexOf('c', 10, 3); // (4) StringIndexOutOfBoundsException
```

Example C.1 finds the number of times (`count`) a substring (`smiley`) occurs in the open range `[beginIndex, endIndex)` of a string (`text`). It uses an infinite loop to find successive occurrences of the substring `smiley` by incrementing the local variable `lastIndex` by the length of the substring if an occurrence is found so that the next search starts after the current occurrence:

Click here to view code image

```
lastIndex = text.indexOf(smiley, lastIndex, endIndex);
```

The loop terminates when the method returns `-1` to indicate that no occurrence was found.

Example C.1 *Searching for a Substring in an Open Range of a String*

Click here to view code image

```
package search;

public class SearchStr {
  public static void main(String[] args) {
    String text = "xxxx:-)xxxx:-)xxx:-(xxx:-)xx";
    //            01234567890123456789012345678
    String smiley = ":-)";
    int beginIndex = 5;
    int endIndex = 26;

    int count = 0;
    int lastIndex = beginIndex;
    int subStrLen = smiley.length();
    while(true) {
      System.out.println("Begin at index: " + lastIndex);
      lastIndex = text.indexOf(smiley, lastIndex, endIndex);
```

```
    if (lastIndex == -1) break;
    System.out.println("Found at index: " + lastIndex);
    count++;
    lastIndex += subStrLen;
   };
   System.out.println("Number of occurrences: "+ count);
  }
 }
```

Output from the program:

Click here to view code image

```
Begin at index: 5
Found at index: 11
Begin at index: 14
Found at index: 23
Begin at index: 26
Number of occurrences: 2
```

java.lang.String

Click here to view code image

```
int indexOf(int ch, int beginIndex, int endIndex)
int indexOf(String str, int beginIndex, int endIndex)
```

Returns the index within this string of the first occurrence of the specified character `ch` or the specified substring `str`, starting the search at the start index `beginIndex` (*included*) and stopping before the stop index `endIndex` (*excluded*)—that is, in the *open range* `[beginIndex, endIndex)`. If there is no such occurrence, it returns `-1`.

The second method returns the same result as the expression:

Click here to view code image

```
s.substring(beginIndex, endIndex).indexOf(str) + beginIndex
```

If `beginIndex` is negative, or `endIndex` is larger than the length of this `String` object, or `beginIndex` is larger than `endIndex`, a `StringIndexOutOfBoundsException` is thrown.

Extending a String Builder

Two new methods have been added to the `StringBuffer` class since Java 17 that allow a string builder to be extended by repeating a codepoint or a character sequence a fixed number of times.

A codepoint is a unique integer value that designates a specific character in the Unicode standard. Its representation in bits depends on the given character encoding. Characters in Java strings are represented in UTF-16 encoding.

All values of type `char` in Java correspond to codepoint values from `U+0000` to `U+FFFF` and are represented by 2 bytes (16 bits) in UTF-16, and codepoints above `U+FFFF` require 4 bytes. Casting a Java `char` value to `int` thus represents the code-point of the `char` value. The `char` value `'|'` passed in the call to the `repeat()` method below is converted to an `int` representing the codepoint of the character.

Click here to view code image

```
StringBuffer accordion = new StringBuffer("[:").repeat('|', 10).append(":]");
System.out.println(accordion);       // [:||||||||||:]
```

Example C.2 prints a frequency histogram of entries in a frequency map. The keys and values in the frequency map are of type `Character` and `Integer`, respectively. At (1), an anonymous class extends the `TreeMap` class and the instance created is populated by the initializer code, inserting 3 entries into the map.

Histogram is printed by iterating over the entries in the map, as shown at (2). An empty string buffer is extended by calling the `repeat()` method, passing the value corresponding to a key in an entry as the number of copies required of the string `"*"`. It is instructive to compare the `StringBuffer.repeat(CharSequence, int)` method, `String.repeat(int)` method, and the static `Collections.nCopies(int, T)` method.

Example C.2 *Repeatedly Appending a Character Sequence to a String Buffer*

Click here to view code image

```
package repeat;

import java.util.Map;
import java.util.TreeMap;

public class Histogram {
  public static void main(String[] args) {

    Map<Character, Integer> freqMap = new TreeMap<>() {           // (1)
     { put('c', 7); put('d', 2); put('a', 4); }
    };
    System.out.println("Frequencies: " + freqMap);

    System.out.println("Histogram:");
    freqMap.forEach((key, value) ->                               // (2)
       System.out.printf("%3s: %s%n", key,
                   new StringBuffer().repeat("*", value)));  // (3)
  }
}
```

Output from the program:

Click here to view code image

```
Frequencies: {a=4, c=7, d=2}
Histogram:
  a: ****
  c: *******
  d: **
```

java.lang.StringBuilder

Click here to view code image

```
StringBuilder repeat(int codePoint, int count)
StringBuilder repeat(CharSequence cs, int count)
```

The first method appends `count` copies of the string representation of the `code-Point` argument to this sequence and returns the result.

The second method appends `count` copies of the specified `CharSequence cs` to this sequence and returns the result.

The length of this sequence increases by `count` * length of string representation of `codePoint` or the `CharSequence cs`, respectively.

If the `CharSequence cs` is `null`, then the four characters `"null"` are repeated into this sequence by the second method.

If the specified `codePoint` is not a valid Unicode code point or if `count` is negative, an `IllegalArgumentException` is thrown.

C.2 Constructing Locales

All constructors of the `java.util.Locale` class are now *deprecated* and therefore should not be used:

Click here to view code image

```
Locale(String language)                          // Deprecated!
Locale(String language, String country)          // Deprecated!
Locale(String language, String country, String variant)  // Deprecated!
```

It is recommended to use the overloaded `of()` methods provided by the API to obtain a `Locale`. The overloaded `of()` methods are analogous to the deprecated constructors.

java.util.Locale

Click here to view code image

```
static Locale of(String language)
static Locale of(String language, String country)
static Locale of(String language, String country, String variant)
```

These static methods obtain a locale from a `language` code, `country` code, and a `variant`, respectively. They normalize the `language` value to lowercase and the `country` value to uppercase.

They also does not make any syntactic checks on the input, and throw a `NullPointerException` if any argument is `null`.

Click here to view code image

```
static Stream<Locale> availableLocales()
```

Returns a stream of installed locales.

Example C.3 uses the `of()` method of the `Locale` class to construct locales, as shown at (1) and (2). The language and country names are intentionally specified in the wrong case, since these are normalized by the `of()` method as can be seen in the printout from the code at (2). The code at (3) displays the country and the language names of the Norwegian locale in different locales.

The `availableLocales()` method of the `Locale` class can be used to obtain a stream of available locales, as shown at (4).

Example C.3 *Creating Locales using the Overloaded* `Locale.of()` *Method*

Click here to view code image

```
package locales;

import java.util.Locale;
public class CreatingLocales {

  public static void main(String[] args) {

    Locale locDF = Locale.getDefault();
    Locale locNO = Locale.of("NO", "no");      // (1) Locale: Norwegian/Norway
    Locale locFR = Locale.of("FR", "fr");      // (2) Locale: French/France

  // Print country and language names:                                    (2)
  System.out.printf("Default locale:  %s (%s)%n",
                    locDF.getCountry(), locDF.getLanguage());
  System.out.printf("Norwegian locale: %s (%s)%n",
                    locNO.getCountry(), locNO.getLanguage());
  System.out.printf("French locale:  %s (%s)%n",
                    locFR.getCountry(), locFR.getLanguage());
```

```
    System.out.println("Display country (language) for Norwegian locale:");      // (3)
    Locale[] locales = {locDF, locNO, locFR};
    for (Locale locale : locales) {
      System.out.printf("In %-13s: %-7s (%s)%n", locale.getDisplayCountry(),
          locNO.getDisplayCountry(locale), locNO.getDisplayLanguage(locale));
    }

    System.out.println("Available locales:");                                  // (4)
    Locale.availableLocales().forEach(l -> System.out.println(l));
    }
}
```

Output from the program:

Click here to view code image

```
Default locale:   US (en)
Norwegian locale: NO (no)
French locale:    FR (fr)
Display country (language) for Norwegian locale:
In United States: Norway  (Norwegian)
In Norway       : Norge   (norsk)
In France       : Norvège (norvégien)
Available locales:
...
```

Previous chapter

Appendix B. Annotated Answers to Review Questions

Next chapter

Part I: Index